

# OOPJ — Day 14

Tuesday, 22 September 2026 | Functional Interfaces, Stream API, Optional, Enums

## 1. Lambda Expressions: Syntax and Conciseness

Lambda expressions allow you to write anonymous functions inline, without creating a separate class. The full syntax includes parameter types, round brackets, an arrow, and curly braces, but Java allows you to drop each of these when they are not needed.

FULL SYNTAX

```
(String str) -> { System.out.println(str); }
```

SIMPLIFIED — TYPE INFERENCE, NO BRACKETS FOR SINGLE PARAM, NO BRACES FOR SINGLE STATEMENT

```
str -> System.out.println(str)
```

Rules:

- Round brackets around the parameter are optional when there is exactly one argument.
- Curly braces around the body are optional when there is exactly one statement.
- If the body is a single expression that returns a value, the `return` keyword is also dropped:

```
str -> str.length()
```

**Why lambdas exist:** Before Java 8, passing behaviour (e.g., a sort comparator) required creating a full named class or an anonymous class. Lambdas eliminate that boilerplate entirely, making code shorter and more readable.

## 2. Lambda vs. Anonymous Class

A common exam and interview question. The key differences:

| Aspect                    | Anonymous Class                           | Lambda Expression                    |
|---------------------------|-------------------------------------------|--------------------------------------|
| <b>What it is</b>         | A class without a name                    | A function without a name            |
| <b>Inheritance</b>        | Inherited from abstract or concrete class | Mapped to a functional interface     |
| <b>Instance variables</b> | Can define instance variables (heap)      | Cannot; only local variables (stack) |
| <b>this keyword</b>       | Refers to the anonymous class object      | Refers to the enclosing class        |
| <b>Boilerplate</b>        | High — must override method explicitly    | None — body is the implementation    |

ANONYMOUS CLASS EXAMPLE (OLD WAY)

```
TreeSet<Employee> ts = new TreeSet<>(new EmployeeNameComparator());  
// Required a separate class implementing Comparator
```

LAMBDA EQUIVALENT (NEW WAY)

```
TreeSet<Employee> ts = new TreeSet<>(  
    (emp1, emp2) -> emp1.getName().compareTo(emp2.getName())  
);
```

No separate `EmployeeNameComparator` class needed. The lambda body directly provides the implementation of the single abstract method `compare()` in the Comparator functional interface.

### 3. Standard Functional Interfaces (java.util.function)

Java 8 introduced the `java.util.function` package, which provides ready-made functional interfaces for the most common lambda patterns. You do not need to define your own functional interface for these standard cases.

#### Predicate<T>

- Use when: you want to **test a condition** and get back a boolean.
- Abstract method: `boolean test(T t)`
- Example: check if a number is even.

```
Predicate<Integer> isEven = n -> n % 2 == 0;  
System.out.println(isEven.test(4)); // true
```

#### Function<T, R>

- Use when: you want to **take an input and return a transformed output**.
- Abstract method: `R apply(T t)`
- Two generic types: T for input, R for return value.
- Example: get the length of a string.

```
Function<String, Integer> getLength = str -> str.length();  
System.out.println(getLength.apply("Hello")); // 5
```

#### Supplier<T>

- Use when: you want to **return a value but pass no argument**.
- Abstract method: `T get()`
- Example: supply a default greeting.

```
Supplier<String> greeting = () -> "Hello, World!";  
System.out.println(greeting.get());
```

## Consumer<T>

- Use when: you want to **accept a value but return nothing** (void).
- Abstract method: `void accept(T t)`
- Example: `System.out.println` is effectively a consumer — it takes a value and prints it, returning nothing.

```
Consumer<String> printer = str -> System.out.println(str);
printer.accept("Java 8");
```

## 4. Bi-Variants of Functional Interfaces

When you need to pass **two arguments** instead of one, Java provides Bi-variants:

- `BiPredicate<T, U>` — tests a condition on two arguments, returns boolean.
- `BiFunction<T, U, R>` — takes two arguments, returns a value.
- `BiConsumer<T, U>` — takes two arguments, returns void.

There is **no BiSupplier** because a Supplier returns a value and takes no arguments — you cannot return two values, so a bi-version makes no sense.

## 5. Primitive-Specific Variants

The generic functional interfaces use object types (e.g., `Integer`, `Double`), which involves autoboxing overhead. For performance-critical code, Java provides primitive-specific variants:

- `IntPredicate` — takes a primitive int, returns boolean.
- `IntFunction<R>` — takes a primitive int, returns R.
- `IntConsumer` — takes a primitive int, returns void.
- `IntSupplier` — returns a primitive int.
- Similarly: `LongPredicate`, `DoublePredicate`, `DoubleFunction`, etc.

EXAMPLE — INTPREDICATE TO CHECK EVEN/ODD

```
IntPredicate isEven = n -> n % 2 == 0;
System.out.println(isEven.test(7)); // false
```

No generic type needed — the parameter is directly a primitive int.

## 6. Using Lambdas with Comparator for TreeSet Sorting

Before lambdas, sorting a `TreeSet` of user-defined objects required a separate comparator class:

```
// Old way - required a separate EmployeeNameComparator class
TreeSet<Employee> ts = new TreeSet<>(new EmployeeNameComparator());
```

With lambdas, since `Comparator` is a functional interface (one abstract method: `compare`), you can pass a lambda directly:

```
TreeSet<Employee> ts = new TreeSet<>(  
    (emp1, emp2) -> emp1.getName().compareTo(emp2.getName())  
);
```

`String.compareTo()` returns:

- 0 if both strings are equal
- 1 (positive) if the first is greater
- -1 (negative) if the first is smaller

This performs natural alphabetical sorting by employee name, with zero boilerplate.

USING `COMPARATOR.NATURALORDER().REVERSED()`

```
Comparator<Employee> comp = Comparator.naturalOrder();  
Comparator<Employee> reversed = comp.reversed();  
TreeSet<Employee> ts = new TreeSet<>(reversed);
```

## 7. Stream API: Core Concepts and Why It Exists

The **Stream API** was introduced in **Java 8** to provide a clean, functional way to process data stored in collections. Without streams, filtering or transforming a list required manual iteration:

```
// Old way - filter names starting with "K" and store in a new list  
List<String> result = new ArrayList<>();  
for (String name : trainees) {  
    if (name.startsWith("K")) {  
        result.add(name);  
    }  
}
```

With the Stream API, the same logic becomes a single pipeline:

```
trainees.stream()  
    .filter(str -> str.startsWith("K"))  
    .forEach(System.out::println);
```

### Key definitions:

- A **stream** is a pipeline for processing data from a source (collection or file).
- Streams **do not store data** — collections store data; streams process it.
- Streams **do not mutate the original collection** — the source list remains unchanged.
- Streams are **single-use** — once a terminal operation is called, the stream is consumed and cannot be reused.

### Three stages of a stream pipeline:

1. **Source** — a collection (List, Set, Map) or file.

2. **Intermediate operations** — lazy operations (filter, map, sorted, etc.) that return a new stream.
3. **Terminal operation** — triggers execution (forEach, collect, reduce, etc.).

**Lazy evaluation:** Intermediate operations do not execute until a terminal operation is invoked. This is why they are also called *lazy operations*.

**Streams only work on the Collection Framework.** Arrays do not natively support `.stream()`. To use streams on an array:

```
int[] data = {1, 5, 2, 7};

// Option 1
Arrays.stream(data).forEach(System.out::println);

// Option 2 - convert to list first
List<Integer> list = Arrays.asList(1, 5, 2, 7);
list.stream().forEach(System.out::println);
```

Internally, `Arrays.stream()` converts the array to a list before providing the stream.

## 8. Intermediate Operations

Each intermediate operation returns a new Stream object, allowing you to chain multiple operations.

### filter(Predicate)

Selects elements that match a condition. Only accepts a Predicate (not a Function).

```
// Filter trainees whose names start with "K"
trainees.stream()
    .filter(str -> str.startsWith("K"))
    .forEach(System.out::println);

// Filter trainees whose name length is greater than 5
trainees.stream()
    .filter(str -> str.length() > 5)
    .forEach(System.out::println);
// Output: only "Aishwarya" (length > 5)
```

### map(Function)

Transforms each element. Accepts a Function — something goes in, something (possibly different) comes out.

```
// Convert all trainee names to uppercase
trainees.stream()
    .map(str -> str.toUpperCase())
    .forEach(System.out::println);

// Chain filter + map
trainees.stream()
```

```
.filter(str -> str.startsWith("K"))
.map(str -> str.toUpperCase())
.forEach(System.out::println);
```

## sorted(Comparator)

Sorts elements. For standard types, natural order is used automatically. For user-defined types, a Comparator must be provided.

```
// Sort books by publication year using ternary comparator
bookList.stream()
    .sorted((book1, book2) ->
        book1.getPublicationYear() > book2.getPublicationYear() ? 1 :
        book1.getPublicationYear() < book2.getPublicationYear() ? -1 : 0)
    .forEach(System.out::println);
// Output: 2016, 2025, 2026, 2026
```

## distinct()

Returns only unique elements. For primitive/String streams this works automatically. For user-defined objects, you **must override hashCode() and equals()** in the class.

```
// Distinct integers
List<Integer> allNumbers = Arrays.asList(1, 5, 2, 7, 5, 9, 4, 2, 9);
allNumbers.stream()
    .distinct()
    .forEach(System.out::println);
// Output: 1, 5, 2, 7, 9, 4
```

For user-defined objects like `Book`, `distinct()` will not work correctly unless `hashCode()` and `equals()` are overridden (see the hashCode/equals section below).

## limit(n)

Caps the stream to the first n elements. Useful for pagination.

```
trainees.stream()
    .limit(3)
    .forEach(System.out::println);
// Returns only the first 3 trainees
```

## peek(Consumer)

Passes each element through without consuming the stream. Primarily used for debugging intermediate steps.

# 9. Terminal Operations

Terminal operations trigger the execution of the entire pipeline. Once called, the stream is consumed and **cannot be reused**.

## Illegal State Exception — Stream Reuse

---

```
Stream<Book> bookStream = bookList.stream();
bookStream.filter(b -> b.getAuthor().startsWith("R"))
    .forEach(System.out::println); // Terminal operation — stream consumed

bookStream.filter(b -> b.getGenre() == BookGenre.TECHNOLOGY)
    .forEach(System.out::println);
// THROWS IllegalStateException
// "Stream has already been operated upon or closed"
```

## forEach(Consumer)

---

Iterates and consumes each element. Once used, the stream is done.

```
trainees.stream()
    .filter(str -> str.startsWith("K"))
    .forEach(System.out::println);
```

## collect(Collectors...)

---

Converts the stream result into a collection object. Unlike `forEach`, the result is stored and can be reused.

```
// Collect filtered names into a List
List<String> filteredNames = trainees.stream()
    .filter(str -> str.startsWith("K"))
    .collect(Collectors.toList());

// Now filteredNames can be iterated multiple times
filteredNames.forEach(System.out::println);
System.out.println(filteredNames); // Can print again
```

### forEach vs. collect:

- `forEach` — consume once, cannot reuse result.
- `collect` — store result in a collection, reuse as many times as needed.

## max(Comparator) and min(Comparator)

---

Returns the maximum or minimum element based on a comparator, wrapped in an `Optional`.

```
Optional<Book> maxBook = bookList.stream()
    .max((book1, book2) ->
        book1.getTotalQuantity() > book2.getTotalQuantity() ? 1 :
        book1.getTotalQuantity() < book2.getTotalQuantity() ? -1 : 0);

if (maxBook.isPresent()) {
    Book data = maxBook.get();
    System.out.println(data);
}
```

## allMatch(Predicate) and anyMatch(Predicate)

Return a boolean based on whether elements match a condition.

```
// Do ALL books have quantity > 20?
boolean allAbove20 = bookList.stream()
    .allMatch(b -> b.getTotalQuantity() > 20);
// Returns false if even one book has quantity <= 20

// Does ANY book have quantity > 20?
boolean anyAbove20 = bookList.stream()
    .anyMatch(b -> b.getTotalQuantity() > 20);
// Returns true if at least one book matches
```

## 10. Collectors Class: Key Methods

`Collectors` is a supporting helper class in `java.util.stream` that provides implementations for the `collect()` terminal operation.

### Collectors.toList()

Returns an ordered list that permits duplicates.

```
List<Book> techBooks = bookList.stream()
    .filter(b -> b.getBookGenre() == BookGenre.TECHNOLOGY)
    .collect(Collectors.toList());
```

- Prior to Java 10: returns a modifiable `ArrayList`.
- Java 16+: returns an **unmodifiable** list (cannot add/remove elements).

### Collectors.toSet()

Returns a Set object (by default a `HashSet`) — no duplicates, no guaranteed order.

```
Set<String> uniqueAuthors = bookList.stream()
    .map(Book::getAuthor)
    .collect(Collectors.toSet());
```

### Collectors.groupingBy(Function)

Groups elements by a key derived from a function. Returns `Map<Key, List<T>>`.

```
Map<BookGenre, List<Book>> booksByGenre = bookList.stream()
    .collect(Collectors.groupingBy(Book::getBookGenre));

// Retrieve all technology books
List<Book> techBooks = booksByGenre.get(BookGenre.TECHNOLOGY);
List<Book> fictionBooks = booksByGenre.get(BookGenre.FICTION);
List<Book> mythologyBooks = booksByGenre.get(BookGenre.MYTHOLOGY);

techBooks.forEach(System.out::println);
```



```
fictionBooks.forEach(System.out::println);
mythologyBooks.forEach(System.out::println);
```

With the sample data (Core Java, Harry Potter, Enterprise Java, .NET book), this produces three groups: TECHNOLOGY (3 books), FICTION (1 book), MYTHOLOGY (0 books).

## Collectors.partitioningBy(Predicate)

Splits the stream into **exactly two groups** based on a true/false condition. Returns `Map<Boolean, List<T>>`.

```
Map<Boolean, List<Book>> partition = bookList.stream()
    .collect(Collectors.partitioningBy(b -> b.getTotalQuantity() >= 100));

List<Book> above100 = partition.get(true);
List<Book> below100 = partition.get(false);

System.out.println("Books with QTY >= 100:");
above100.forEach(System.out::println);

System.out.println("Books with QTY < 100:");
below100.forEach(System.out::println);
```

### Difference between filter and partitioningBy:

- filter returns only the matching elements (true side only).
- partitioningBy returns both groups (true and false) in a single map.

## 11. Optional Class

`Optional<T>` is a utility class introduced in Java (around version 7–8) to handle the possibility of a null return value gracefully, avoiding `NullPointerException`.

### PROBLEM WITHOUT OPTIONAL

```
Employee emp = getData(); // What if getData() returns null?
emp.getBasicSalary();      // NullPointerException!
```

### SOLUTION WITH OPTIONAL

```
Optional<Employee> opt = getData(); // Returns Optional, never null itself

if (opt.isPresent()) {
    Employee emp = opt.get();
    System.out.println(emp.getBasicSalary());
}

// Or use orElseThrow for stricter handling
Employee emp = opt.orElseThrow(() -> new RuntimeException("No employee found"));
```

### Key methods:

- `isPresent()` — returns true if a value is present.

- `isEmpty()` — returns true if no value is present.
- `get()` — retrieves the value (throws exception if empty).
- `orElseThrow(Supplier)` — retrieves value or throws a specified exception.

In the Stream API, `max()` and `min()` return `Optional<T>` because the stream might be empty, in which case there is no maximum or minimum to return.

## 12. Enums

An **enum** (enumeration) is used whenever you need a **fixed, predefined set of values** for a field. Without an enum, using a String for something like book genre allows any arbitrary value to be set, including typos or invalid entries.

### PROBLEM WITH STRING

```
book.setGenre("Technology"); // Typo - no compile-time error
book.setGenre("aesthetic"); // Invalid - no compile-time error
```

### SOLUTION WITH ENUM

```
public enum BookGenre {
    TECHNOLOGY,
    FICTION,
    CRIME_THRILLER,
    MYTHOLOGY,
    SELF_HELP
}
```

Now the field is **type-safe** — only valid constants can be assigned.

### Rules for enums:

- An enum is essentially a class, but **cannot be inherited from any other class** (it already inherits from `java.lang.Enum`).
- An enum **can implement interfaces**.
- Constants must be declared **first**, before any fields or methods.
- Can have fields, constructors, and methods.

### ENUM WITH FIELDS AND CONSTRUCTOR (EXAMPLE FROM LECTURE)

```
public enum BookGenre {
    TECHNOLOGY(2025),
    FICTION(1987),
    CRIME_THRILLER(1800),
    MYTHOLOGY(1500);

    private final int releaseYear;

    BookGenre(int releaseYear) {
        this.releaseYear = releaseYear;
    }

    public int getReleaseYear() {
```

```
        return releaseYear;
    }
}
```

### Built-in enum methods:

- `BookGenre.values()` — returns an array of all constants.
- `BookGenre.valueOf("FICTION")` — converts a String to the enum constant.

### USING ENUM IN THE BOOK CLASS

```
Book b = new Book(1000, "Core Java", "Pankaj", BookGenre.TECHNOLOGY, 2026, 100);
```

## 13. The Book / Library Service Example

This was the main practical example used to demonstrate all Stream API features together.

### BOOK POJO

```
public class Book {
    private int bookId;
    private String bookName;
    private String bookAuthor;
    private BookGenre bookGenre;
    private int publicationYear;
    private int totalQuantity;

    // Constructor, getters, setters, toString
}
```

### SAMPLE DATA

```
List<Book> bookList = Arrays.asList(
    new Book(1000, "Core Java", "Pankaj", BookGenre.TECHNOLOGY, 2026, 100),
    new Book(1001, "Harry Potter", "Rylan", BookGenre.FICTION, 2016, 50),
    new Book(1002, "Enterprise Java", "Pankaj", BookGenre.TECHNOLOGY, 2026, 90),
    new Book(1003, ".NET Book", "Rahul", BookGenre.TECHNOLOGY, 2025, 500)
);
```

### FILTER BY AUTHOR NAME STARTING WITH "R"

```
bookList.stream()
    .filter(book -> book.getBookAuthor().startsWith("R"))
    .forEach(System.out::println);
// Output: Harry Potter (Rylan), .NET Book (Rahul)
```

### FILTER BY GENRE (USING ENUM)

```
List<Book> techBooks = bookList.stream()
    .filter(book -> book.getBookGenre() == BookGenre.TECHNOLOGY)
    .collect(Collectors.toList());
// Output: 3 books (Core Java, Enterprise Java, .NET Book)
```

### SORT BY PUBLICATION YEAR

```
bookList.stream()
    .sorted((book1, book2) ->
        book1.getPublicationYear() > book2.getPublicationYear() ? 1 :
        book1.getPublicationYear() < book2.getPublicationYear() ? -1 : 0)
    .forEach(System.out::println);
// Output order: 2016, 2025, 2026, 2026
```

#### MAX BY TOTAL QUANTITY

```
Optional<Book> maxBook = bookList.stream()
    .max((book1, book2) ->
        book1.getTotalQuantity() > book2.getTotalQuantity() ? 1 :
        book1.getTotalQuantity() < book2.getTotalQuantity() ? -1 : 0);

if (maxBook.isPresent()) {
    System.out.println(maxBook.get());
}
// Output: .NET Book (quantity 500)
```

## 14. hashCode() and equals() — Required for distinct()

Every Java class inherits two methods from `java.lang.Object` :

- `hashCode()` — returns an integer hash value for the object.
- `equals(Object obj)` — returns true if two objects are considered equal.

**Why both must be overridden together:**

- hashCode is checked first for performance (hash-based structures like HashSet, HashMap).
- If hash codes match, equals is called to confirm actual equality.
- If you override only one, hash-based collections and `distinct()` will misbehave.

**Rule:** If two objects contain the same data, they must return the same hashCode.

#### ECLIPSE-GENERATED HASHCODE AND EQUALS FOR BOOK (USING ALL FIELDS)

```
@Override
public int hashCode() {
    return Objects.hash(bookId, bookName, bookAuthor, bookGenre, publicationYear,
        totalQuantity);
}

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;
    Book data = (Book) obj;
    return bookId == data.bookId &&
        Objects.equals(bookName, data.bookName) &&
        Objects.equals(bookAuthor, data.bookAuthor);
}
```

#### USING ONLY BOOKID AS THE UNIQUENESS CRITERION (LIKE A PRIMARY KEY)

```

@Override
public int hashCode() {
    return Objects.hash(bookId);
}

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Book)) return false;
    Book data = (Book) obj;
    return bookId == data.bookId;
}

```

With this, two Book objects with the same `bookId` are considered duplicates by `distinct()`, even if their names differ.

```

bookList.stream()
    .distinct()
    .forEach(System.out::println);
// Duplicates removed based on bookId uniqueness

```

## 15. Parallel Streams

Parallel streams are an extension of the Stream API, also introduced in Java 8, designed to improve performance on **multi-core systems**.

```

bookList.parallelStream()
    .filter(b -> b.getBookGenre() == BookGenre.TECHNOLOGY)
    .forEach(System.out::println);

```

- Internally uses threads and handles synchronization automatically.
- Preferred over sequential streams when processing **large datasets**.
- Not recommended for small datasets — thread management overhead can outweigh the benefit.
- Full understanding requires knowledge of threading and synchronization (covered in a later session).

## 16. Scanner Class

The `Scanner` class (`java.util.Scanner`) is a utility for reading input from the console (or a file). It was introduced as a simpler alternative to the raw `InputStream` byte-array approach used in early Java.

BASIC USAGE

```

Scanner scanner = new Scanner(System.in);

System.out.println("Enter name:");
String name = scanner.next(); // Reads a single token (stops at space)

```

```
System.out.println("Enter year:");
int year = scanner.nextInt(); // Reads an integer

System.out.println(name + " " + year);
scanner.close(); // Always close the scanner - it is a resource
```

**Handling names with spaces:** `scanner.next()` treats a space as a delimiter and stops reading. For multi-word input like “Harry Potter”:

```
String name = scanner.nextLine(); // Reads the entire line including spaces
```

#### EXAMPLE

```
Scanner scanner = new Scanner(System.in);
String name = scanner.nextLine(); // Input: "Harry Potter" → reads full string
int year = scanner.nextInt();
System.out.println(name + ", " + year);
scanner.close();
```

**Why Scanner was not introduced earlier in the course:** It requires understanding of `InputStream` (`System.in`), which was covered as part of the `java.io` package. Once that was understood, Scanner became straightforward.

## 17. Why Default Methods Were Added to Collection (Java 8 Context)

The `stream()` method was added to the `Collection` interface in Java 8. This raised a design challenge: adding a new abstract method to an existing interface would break all existing implementations.

**Solution: default methods.** Java 8 introduced default methods in interfaces — methods with a body that implementing classes inherit automatically without needing to override them.

```
// In Collection interface (Java 8+)
default Stream<E> stream() { ... }
default Stream<E> parallelStream() { ... }
```

This allowed Java to add `stream()` to `Collection` without breaking any existing code that implemented Collection before Java 8.

## 18. Upcoming Topics

- **Threading and Synchronization** — next session (Wednesday); covers how parallel streams work internally.
- **JDBC (Java Database Connectivity)** — scheduled for Thursday, but pending confirmation.
  - Requires prior knowledge of SQL, DDL, DML.

- Instructor recommendation: skip JDBC if the database module has not been covered.
- If JDBC is skipped, threading and synchronization will be the final session of the course.

---

Meeting transcript reference: [notes.granola.ai/t/2787a3be...](https://notes.granola.ai/t/2787a3be...)